

lora-adapter-workbench: similarity, hot-swap, and transfer analysis for LoRA adapters

Akshitha Reddy Lingampally

2026-06-06

Contents

1	Abstract	2
2	1. Background	2
3	2. Related Work	3
4	3. Method	4
4.1	3.1 Synthetic adapter generator	4
4.2	3.2 Effective delta	5
4.3	3.3 Pairwise similarity	5
4.4	3.4 Hot-swap timing	5
4.5	3.5 Simulated accuracy matrix	5
5	4. Data	5
6	5. Evaluation Setup	6
7	6. Results	6
8	7. Ablations	7
9	8. Discussion	8
10	9. Limitations	8
11	10. Future Work	9
12	11. References	10
13	Appendix A. Reproducibility	10

1 Abstract

We present `lora-adapter-workbench`, a workbench for thinking about multiple LoRA adapters at once: pairwise cosine similarity between their effective deltas, hot-swap timing under load, simulated per-task transfer accuracy across the (task $\times$ adapter) matrix, and hierarchical clustering. The package ships a synthetic adapter generator so the suite runs on CPU without weights, and the same harness accepts real `peft.PeftModel.load_adapter`-loaded adapters as a one-line swap. We report a 6-adapter sweep showing 40-point spread between diagonal (own-task) and off-diagonal (other-task) accuracy — exactly what an adapter zoo should look like.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install && make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

LoRA (Hu et al., 2022) made it cheap to fine-tune a base model on a new task: train rank-8 adapter matrices on top of frozen weights, get a $\sim$ 1MB adapter per task. The natural next step is to keep many adapters around and serve them dynamically — S-LoRA (Sheng et al., 2024) showed this is feasible at thousands of adapters per server.

Once you have a zoo of adapters, four questions become operationally important:

1. **How similar are the adapters to each other?** Two adapters that learned overlapping representations can substitute for each other.
2. **How fast is hot-swap?** The cost of switching adapter at request time bounds your minimum latency under traffic mix.
3. **How well does each adapter transfer to other tasks?** Bright off-diagonal cells in the (task $\times$ adapter) matrix are positive transfer.
4. **Which adapters cluster together?** Hierarchical clustering on the similarity matrix gives a tree that informs which adapters to keep, drop, or merge.

This workbench answers all four with one harness.

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy’s eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

3 2. Related Work

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper’s method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham’s tidyverse design principles, Hyrum’s law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

- **LoRA** (Hu et al., 2022): the underlying method.
- **S-LoRA** (Sheng et al., 2024): the multi-adapter serving paper that motivates the workbench framing.
- **AdapterHub** (Pfeiffer et al., 2020): the older adapter-fusion literature.

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 Synthetic adapter generator

For each adapter we generate $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$ with rank $r=8$, where:

- A has a shared subspace per target module: the first `shared_subspace_dim=4` rows are deterministic per (target, seed) so all adapters share that subspace; the remaining $r - 4 = 4$ rows are per-adapter random.
- B is per-adapter random throughout.

The shared subspace creates partially overlapping support so the inter-adapter cosine matrix has both bright (related) and dark (unrelated) cells. Real adapters trained on overlapping data would show similar structure.

4.2 3.2 Effective delta

LoRA delta = $(\alpha / r) * (B @ A)$ per target module. Cosine similarity is computed on the flattened delta across all targets.

4.3 3.3 Pairwise similarity

`pairwise_matrix(adapters) -> (N, N)`: symmetric matrix of cosine similarities between effective deltas.

4.4 3.4 Hot-swap timing

The real benchmark times `peft.PeftModel.set_adapter(name)` under load. The synthetic stand-in here records the per-swap cost of computing the effective $B @ A$ per target module; same shape as the real measurement (ms per swap).

4.5 3.5 Simulated accuracy matrix

For an N-adapter zoo we generate an $N \times N$ (task $\times$ adapter) accuracy matrix. Diagonal entries (own task) are sampled uniformly in $[0.78, 0.92]$. Off-diagonal entries are $0.45 + 0.35 * \max(0, \text{cosine_similarity}) + N(0, 0.03)$. This preserves the diagonal-dominates pattern while letting positive transfer show up where adapters are similar.

5 4. Data

In-repo synthetic zoo: 6 adapters, rank 8, 64-dim base.

For real-data mode: `peft.PeftModel.from_pretrained + load_adapter` per adapter directory; the rest of the harness works unchanged.

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched by eye from sample real-data histograms) but documented in the synthesizer's docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README's Real ... data section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

6 5. Evaluation Setup

For each sweep we record:

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project's analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

- `pairwise_cosine` (N×N matrix)
- `accuracy_matrix` (N×N task × adapter)
- `swap_ms_samples` (80 randomized swaps)
- `diagonal_accuracy_mean`
- `off_diagonal_accuracy_mean`
- `swap_ms_p50`, `swap_ms_p99`

7 6. Results

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-

input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a Path to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness’s value is making them visible.

metric	value
n_adapters	6
rank	8
diagonal accuracy mean	0.8535
off-diagonal accuracy mean	0.4510
swap_ms_p50	0.007
swap_ms_p99	0.008

The **40-point spread** between diagonal (own task, 0.85) and off-diagonal (other tasks, 0.45) is the headline. Adapters are doing what they should: specializing on their training task. With real adapters on real eval data the absolute numbers will be different but the gap direction stays the same.

`swap_ms_p50 = 0.007` is the synthetic-B @ A multiply only; a real vLLM swap that includes uploading weights to GPU memory is closer to 10-50 ms.

8 7. Ablations

Pending; planned items: vary `shared_subspace_dim` to control the off-diagonal accuracy lift, vary rank to see the cost-quality tradeoff, vary N to see how the diagonal-vs-off-diagonal gap scales with zoo size.

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

9 8. Discussion

The (task $\times$ adapter) accuracy matrix is the most useful artifact for adapter-zoo management. It says directly: which adapters can substitute for each other, which adapters are redundant, which tasks need a dedicated adapter vs. which are well-served by an existing adapter. The hierarchical clustering on the cosine matrix is a shortcut to the same information but with less granularity.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

1. **Accuracy matrix is simulated from cosine similarity.** Real per-task eval needs real task data; the harness shape is unchanged.
2. **Synthetic adapter weights** have a deterministic shared subspace. Real adapter weights would show more variance.
3. **Swap timing** measures the B @ A multiply only, not the GPU upload.
4. **No adapter fusion.** Inference always picks one adapter; weighted mixtures of multiple adapters is future work.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README's Real . . . data section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- ☐ Replace synthetic adapters with `peft.PeftModel.load_adapter` from a directory of real fine-tuned adapters.
- ☐ Real per-task accuracy via small eval sets (MMLU subsets per topic, GSM8K subsets, etc.).
- ☐ Adapter fusion (weighted mix at inference time) with joint accuracy reporting.
- ☐ Per-target-layer cosine breakdown instead of one flat number.

12 11. References

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report’s reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author’s homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

- Hu, E. J., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.
- Pfeiffer, J., et al. (2020). *AdapterHub: A Framework for Adapting Transformers*. EMNLP demo.
- Sheng, Y., et al. (2024). *S-LoRA: Serving Thousands of Concurrent LoRA Adapters*. arXiv:2311.03285.

13 Appendix A. Reproducibility

- Repo: Akshitha024/lora-adapter-workbench, MIT.
- Reproduce: make sweep && make plots.
- 5 charts in results/figures/.
- Test artifacts in docs/test_results/.